

Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 4

Disciplina: Tehnici de programare

Tema: ” Introducere în Python ”

A efectuat :

st. gr.SI – 264 Șoimaru Gabriel

A verificat :

asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Completarea cerințelor de pe site-urile oferite.

1. Primele 2 module de pe <https://www.codedex.io/python> (deoarece restul sunt cu plată)

2. Toate modulele de pe <https://www.learnpython.org/>

EFFECTUAREA LUCRĂRII

1. Codedex.io:

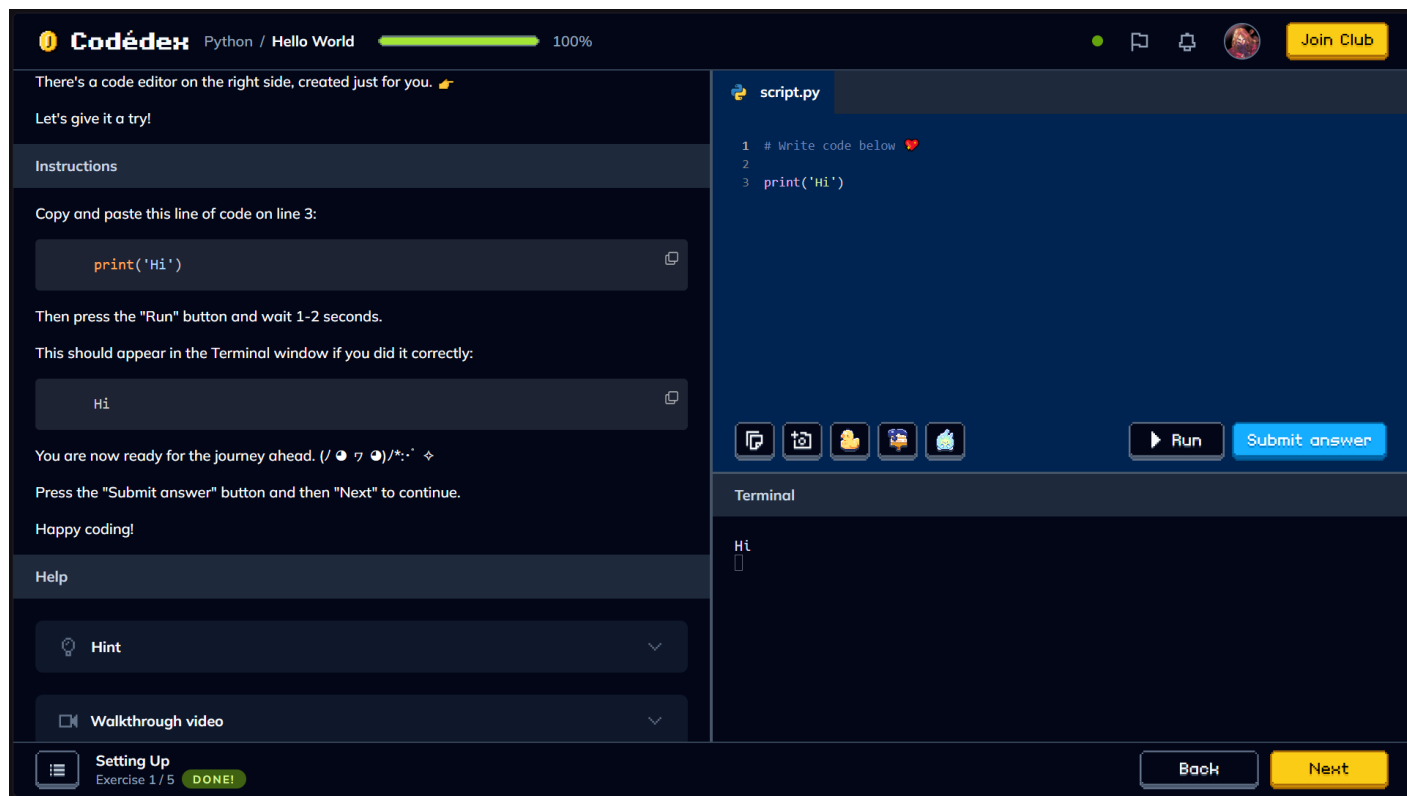


Figura 1.1 - Setting up

Python / Hello World
100%

Join Club

This is an example of a `print()` function:

```
print('👋 Howdy')
```

In the example above, we instructed our program to print a message. The resulting text that is printed to the screen is referred to as the **output**.

The output of this example program would be:

```
👋 Howdy
```

Instructions

Let's use our editor to create a new `hello_world.py` program.

On a new line, use `print()` to output the message "Hello World!"

And then run the program.

Were you able to print "Hello World!" in the terminal?

Help

Hint

Walkthrough video

Hello World
Exercise 2 / 5 DONE!

Back
Next

hello_world.py

```

1 # Write code below ❤️
2
3 print("Hello World!")

```

Run
Submit answer

Terminal

```

Hello World!

```

Figura 1.2 - Hello World

Python / Hello World
100%

Join Club

```

print('🌞 Morning Dharma!')
print('🌃 Evening Sonny!')

```

This will output:

```

🌞 Morning Dharma!
🌃 Evening Sonny!

```

Now let's use what we just learned to complete a special challenge! ✨💎

Instructions

Suppose we want the output to look exactly like this pattern below:

```

  1
 2 3
4 5 6
7 8 9 10

```

How can you do that?

Create a `pattern.py` program that prints this pattern exactly as shown.

This will likely take some trial and error, but give it a shot!

Help

Pattern
Exercise 3 / 5 DONE!

Back
Next

pattern.py

```

1 # Write code below ❤️
2
3 print('    1')
4 print('  2 3')
5 print('4 5 6')
6 print('7 8 9 10')

```

Run
Submit answer

Terminal

```

    1
  2 3
4 5 6
7 8 9 10

```

Figura 1.3 - Pattern

Python / Hello World

100%

Join Club

Here, the output will still be "Hi" because any code before the comment on the same line is still executed.

Instructions

Create an **initials.py** program that displays your initials in **block letters**.

First, start the program with a comment that says a fun fact about yourself.

Then, create your block letters with your initials.

For example, if your name is Dua Lipa, your initials would be D and L, and your block letters would look like this:

```

      DDDD  L
    D  D  L
    D  D  L
    D  D  L
    D  D  L
    D  D  L
    D  D  L
    DDDD  LLLLL
  
```

Help

Hint

Initials
Exercise 4 / 5
DONE!

initials.py

```

1 # Write code below ❤️
2
3 #I live in Chisinau
4 print(' GGGG  SSSS')
5 print('G  G  S  S')
6 print('G      S')
7 print('GGGGGG  SSSS')
8 print('G  G  S  S')
9 print('G  G  S  S')
10 print(' GGGG  SSSS')
  
```

Run
Submit answer

Terminal

```

GGGG  SSSS
G  G  S  S
G  G  S
GGGGGG  SSSS
G  G  S
G  G  S  S
GGGG  SSSS
  
```

Back
Next

Figura 1.4 - Initials

Python / Hello World

100%

Join Club

Create a blank **letter.py** program.

In this exercise, you will write a letter to your future self... in Python.

Take a moment and think about what you hope to achieve on this journey.

Use `print()` to output:

1. Today's date.
2. How you are feeling right now.
3. What you want to accomplish by learning to code.
4. A message to your older, wiser, and *programmer* self.
5. Your favorite emoji to spice things up!

When it's done, take a screenshot and click the Twitter/X icon to post your letter!

P.S. We will return the letter when the time is right. 😊

Share to Twitter/X

Help

Snail Mail
Exercise 5 / 5
DONE!

letter.py

```

1 # Write code below ❤️
2
3 print('22/09/2026')
4 print('Good')
5 print('Learn')
6 print('Sorry')
7 print('😭')
  
```

Run
Submit answer

Terminal

```

22/09/2026
Good
Learn
Sorry
😭
  
```

Back
Next

Figura 1.5 - Snail Mail

Python / Variables
100%
Join Club

```
message = "good nite"
username = '@snoopdogg'
```

When we did printing in the first chapter, we printed a string data type.

Boolean

A Boolean data type, or `bool`, stores a value that can only be either true or false. In Python, it's capitalized `True` or `False`. It's named after the British mathematician [George Boole](#) (1815-1864).

```
late_to_class = False
cranky = True
```

Instructions

No instructions in this exercise! You got a freebie. 🍷

Press "Submit answer" and "Next", and go get 'em!

Optional: Create 4 variables in the code editor.

Help

Hint

Data Types

Exercise 1 / 5

DONE!

script.py

```
1 # Write code below ❤️
2
3 date = 22
4 pi = 3.14
5 message = 'nice'
6 tired = True
```

Run

Submit answer

Terminal

Back

Next

Figura 1.6 - Data Types

Python / Variables
100%
Join Club

Another way to write this is using parentheses to calculate the total and the tip in one line:

```
tip = (pizza + coke) * 0.2
```

In Python, parentheses have the highest order of operations, just like [PEMDAS](#).

Now that we've learned about the basics of variables and operators, let's put them to use!

Instructions

Create a **temperature.py** program that converts a number from Fahrenheit (°F) to Celsius (°C).

Google the current temperature of [Brooklyn, NY](#) (where Codédex is based) in °F.

Use the following formula and write it out in Python:

$$Celsius = \frac{(Fahrenheit - 32)}{1.8}$$

Print out the converted temperature. 🚫

Help

Hint

Temperature

Exercise 2 / 5

DONE!

temperature.py

```
1 # Write code below ❤️
2
3 F = 62
4 C = (F-32)/1.8
5 print(C)
```

Run

Submit answer

Terminal

```
16.666666666666668
```

Back

Next

Figura 1.7 - Temperature

Python / Variables

100%

Let's give it a try!

Instructions

The **body mass index (BMI)** was created by a Belgian mathematician in the 1850s and it's used by health and nutrition professionals to quickly estimate body fat in certain populations.

The formula is pretty simple: Take an individual's weight (mass) in kilograms and dividing it by their height in meters, squared.

$$bmi = \frac{mass}{height^2}$$

Create a **bmi.py** program that calculates your own BMI.

Author's note: Psst. BMI is an archaic and oversimplified way to measure personal health. It was created by a mathematician – not a doctor! 💡

Help

Hint

Solution

BMI

Exercise 3 / 5

DONE!

Back

Next

bmi.py

```

1 # Write code below ❤️
2
3 mass = 80
4 height = 1.8
5 bmi = mass/(height ** 2)
6 print(bmi)

```

Run

Submit answer

Terminal

```

24.691358024691358

```

Figura 1.8 - BMI

Python / Variables

100%

Now when the user types 24 and presses `enter`, the `age` variable will be an integer 24, not a text string "24".

Instructions

If you slept through your geometry class, a **Pythagorean theorem** is the relationship between the three sides of a right triangle. It was named after the Greek philosopher Pythagoras, born around 570 BC.

Pythagorean equation looks like:

$$c = \sqrt{a^2 + b^2}$$

- The *a* is the length of a short side.
- The *b* is the length of another short side.
- The *c* is the length of the hypotenuse.

The hypotenuse is the longest side of the right triangle.

Create a **hypotenuse.py** program that asks the user for two numbers, *a* and *b*, and then calculates the hypotenuse *c*. See the hint on how to do square root!

Bonus: If you are looking for a harder challenge, try the [Quadratic formula](#)!

Help

Pythagorean Theorem

Exercise 4 / 5

DONE!

Back

Next

hypotenuse.py

```

1 # Write code below ❤️
2
3 a = int(input("Care este lungimea catetei a: "))
4 b = int(input("Care este lungimea catetei b: "))
5 c = ((a ** 2)+(b ** 2)) ** 0.5
6 print(f"Ipotenuza este {c}")

```

Run

Submit answer

Terminal

```

Care este lungimea catetei a: 4
Care este lungimea catetei b: 3
Ipotenuza este 5.0

```

Figura 1.9 - Pythagorean

CodédexPython / Variables100%

Let's put it all together now!

Instructions

We just got home from a fun trip to South America, specifically Colombia, Peru, and Brazil. There's some leftover cash in:

- `co` Colombian pesos
- `PE` Peruvian soles
- `BR` Brazilian reais

Create a `currency.py` program that asks the user for the amount they have in pesos, soles, and reais and calculates the *total* in USD.

Make sure to Google the current exchange rates for `pesos`, `soles`, and `reais`!



The output should look something like this, but with different results:

```
What do you have left in pesos? 5600
What do you have left in soles? 105
What do you have left in reais? 280
```

currency.py

```
1 # Write code below
2
3 pesos = int(input("What do you have left in pesos? "))
4 soles = int(input("What do you have left in soles? "))
5 reais = int(input("What do you have left in reais? "))
6 total = (pesos * 0.00032) + (soles * 0.30) + (reais * 0.20)
7 print(f"You have {total} left in dollars.")
```

RunSubmit answer

Terminal

```
What do you have left in pesos? 5600
What do you have left in soles? 105
What do you have left in reais? 280
You have 89.292 left in dollars.
```

Currency

Exercise 5 / 5

DONE!

BackNext

Figura 1.10 - Currency

2. Learnpython.org:

2.1. Learn the Basics

Exercise

Use the "print" function to print the line "Hello, World!".

script.py

1 print("Hello, World!")

IPython Shell

<script.py> output:
Hello, World!

In [1]:

Great job!

Solution

Submit

Figura 2.1.1 - Hello, World!

The target of this exercise is to create a string, an integer, and a floating point number. The string should be named `mystring` and should contain the word "hello". The floating point number should be named `myfloat` and should contain the number 10.0, and the integer should be named `myint` and should contain the number 20.

script.py

1 # change this code
2 mystring = "hello"
3 myfloat = 10.0
4 myint = 20
5
6 # testing code
7 if mystring == "hello":
8 print("String: %s" % mystring)
9 if isinstance(myfloat, float) and myfloat == 10.0:
10 print("Float: %f" % myfloat)
11 if isinstance(myint, int) and myint == 20:
12 print("Integer: %d" % myint)

IPython Shell

<script.py> output:
String: hello
Float: 10.000000
Integer: 20

In [1]:

Great job!

Solution

Submit

Figura 2.1.2 - Variables and Types

In this exercise, you will need to add numbers and strings to the correct lists using the "append" list method. You must add the numbers 1,2, and 3 to the "numbers" list, and the words 'hello' and 'world' to the strings variable.

You will also have to fill in the variable `second_name` with the second name in the names list, using the brackets operator `[]`. Note that the index is zero-based, so if you want to access the second item in the list, its index will be 1.

script.py

```
1 numbers = []
2 strings = []
3 names = ["John", "Eric", "Jessica"]
4
5 # write your code here
6 second_name = names[1]
7 numbers.append(1)
8 numbers.append(2)
9 numbers.append(3)
10 strings.append('hello')
11 strings.append('world')
12
13 # this code should write out the filled arrays and
14 # the second name in the names list (Eric).
15 print(numbers)
16 print(strings)
```

IPython Shell

<script.py> output:
[1, 2, 3]
['hello', 'world']
The second name on the names list is Eric

In [1]: |

Great Job!

Solution Submit

Figura 2.1.3 - Lists

Exercise

The target of this exercise is to create two lists called `x_list` and `y_list`, which contain 10 instances of the variables `x` and `y`, respectively. You are also required to create a list called `big_list`, which contains the variables `x` and `y`, 10 times each, by concatenating the two lists you have created.

script.py

```
1 x = object()
2 y = object()
3
4 # TODO: change this code
5 x_list = [x] * 10
6 y_list = [y] * 10
7 big_list = x_list + y_list
8
9 print("x_list contains %d objects" % len(x_list))
10 print("y_list contains %d objects" % len(y_list))
11 print("big_list contains %d objects" % len(big_list))
12
13 # testing code
14 if x_list.count(x) == 10 and y_list.count(y) == 10:
```

IPython Shell

<script.py> output:
x_list contains 10 objects
y_list contains 10 objects
big_list contains 20 objects
Almost there...
Great!

In [1]: |

Good work!

Solution Submit

Figura 2.1.4 - Basic Operators

Exercise

You will need to write a format string which prints out the data using the following syntax: `Hello John Doe. Your current balance is $53.44.`

script.py

```
1 data = ("John", "Doe", 53.44)
2 format_string = "Hello %s %s. Your current balance
  is %s."
3
4 print(format_string % data)
```

IPython Shell

```
<script.py> output:
Hello John Doe. Your current balance is $53.44

<script.py> output:
Hello John Doe. Your current balance is $

<script.py> output:
Hello John Doe. Your current balance is $53.44.

In [1]: |
```

Great work!

Solution

Submit

Figura 2.1.5 - String Formatting

Exercise

Try to fix the code to print out the correct information by changing the string.

script.py

```
1 s = "Strings are awesome!"
2 # Length should be 20
3 print("Length of s = %d" % len(s))
4
5 # First occurrence of "a" should be at index 8
6 print("The first occurrence of the letter a = %d"
  % s.index("a"))
7
8 # Number of a's should be 2
9 print("a occurs %d times" % s.count("a"))
10
11 # Slicing the string into bits
12 print("The first five characters are '%s'" % s[
  :5]) # Start to 5
13 print("The next five characters are '%s'" % s[5
  :10]) # 5 to 10
```

IPython Shell

```
Length of s = 20
The first occurrence of the letter a = 8
a occurs 2 times
The first five characters are 'Strin'
The next five characters are 'gs ar'
The thirteenth character is 'a'
The characters with odd index are 'tig r wsm!'
The last five characters are 'some!'
String in uppercase: STRINGS ARE AWESOME!
String in lowercase: strings are awesome!
String starts with 'Str'. Good!
String ends with 'ome!'. Good!
Split the words of the string: ['Strings', 'are',
  'awesome!']
```

Great work!

Solution

Submit

Figura 2.1.6 - Basic String Operations

Exercise

Change the variables in the first section, so that each if statement resolves as True.

script.py

```
1 # change this code
2 number = 16
3 second_number = 0
4 first_array = [1,2,3]
5 second_array = [1,2]
6
7 if number > 15:
8     print("1")
9
10 if first_array:
11     print("2")
12
13 if len(second_array) == 2:
14     print("3")
15
16 if len(first_array) + len(second_array) == 5:
```

IPython Shell

<script.py> output:

1
2
3
4
5
6

In [1]:

Great Work!

Solution

Submit

Figura 2.1.7 - Conditions

Loop through and print out all even numbers from the numbers list in the same order they are received. Don't print any numbers that come after 237 in the sequence.

script.py

```
219, 390, 984, 592, 236, 105, 942, 941,
4 386, 462, 47, 418, 907, 344, 236, 375, 823,
566, 597, 978, 328, 615, 953, 345,
5 399, 162, 758, 219, 918, 237, 412, 566, 826,
248, 866, 950, 626, 949, 687, 217,
6 815, 67, 104, 58, 512, 24, 892, 894, 767, 553
, 81, 379, 843, 831, 445, 742, 717,
7 958, 609, 842, 451, 688, 753, 854, 685, 93,
857, 440, 380, 126, 721, 328, 753, 470,
8 743, 527
9 ]
10
11 # your code goes here
12 for number in numbers:
13     if number == 237:
14         break
```

IPython Shell

592
236
942
386
462
418
344
236
566
978
328
162
758
918

Great work!

Solution

Submit

Figura 2.1.8 - Loops

1. Add a function named `list_benefits()` that returns the following list of strings: "More organized code", "More readable code", "Easier code reuse", "Allowing programmers to share and connect code together"
2. Add a function named `build_sentence(info)` which receives a single argument containing a string and returns a sentence starting with the given string and ending with the string " is a benefit of functions!"
3. Run and see all the functions work together!

script.py

```
1 # Modify this function to return a list of strings
  as defined above
2 def list_benefits():
3     return ["More organized code", "More readable
  code", "Easier code reuse", "Allowing programmers
  to share and connect code together"]
4
5 # Modify this function to concatenate to each
  benefit - " is a benefit of functions!"
6 def build_sentence(benefit):
7     return benefit + " is a benefit of functions!"
8
9 def name_the_benefits_of_functions():
10     list_of_benefits = list_benefits()
11     for benefit in list_of_benefits:
12         print(build_sentence(benefit))
```

IPython Shell

```
<script.py> output:
More organized code is a benefit of functions!
More readable code is a benefit of functions!
Easier code reuse is a benefit of functions!
Allowing programmers to share and connect code together
  is a benefit of functions!

In [1]: |
```

Nice work!

Solution

Submit

Figura 2.1.9 - Functions

Exercise

We have a class defined for vehicles. Create two new vehicles called `car1` and `car2`. Set `car1` to be a red convertible worth \$60,000.00 with a name of Fer, and `car2` to be a blue van named Jump worth \$10,000.00.

script.py

```
6 value = 100.00
7 def description(self):
8     desc_str = "%s is a %s %s worth $%.2f." %
  (self.name, self.color, self.kind, self.value)
9     return desc_str
10 # your code goes here
11 car1 = Vehicle()
12 car1.name = "Fer"
13 car1.color = "red"
14 car1.kind = "convertible"
15 car1.value = 60000.00
16
17 car2 = Vehicle()
18 car2.name = "Jump"
19 car2.color = "blue"
20 car2.kind = "van"
21 car2.value = 10000.00
```

IPython Shell

```
<script.py> output:
Fer is a red convertible worth $60000.00.
Jump is a blue van worth $10000.00.

In [1]: |
```

Great job!

Solution

Submit

Figura 2.1.10 - Classes and Objects

Exercise

Add "Jake" to the phonebook with the phone number 938273443, and remove Jill from the phonebook.

script.py

```
1 phonebook = {
2     "John" : 938477566,
3     "Jack" : 938377264,
4     "Jill" : 947662781
5 }
6 # your code goes here
7 phonebook["Jake"] = 938273443
8 phonebook.pop("Jill")
9
10 # testing code
11 if "Jake" in phonebook:
12     print("Jake is listed in the phonebook.")
13
14 if "Jill" not in phonebook:
15     print("Jill is not listed in the phonebook.")
```

IPython Shell

<script.py> output:
Jake is listed in the phonebook.
Jill is not listed in the phonebook.

In [1]: |

Nice work!

Solution

Submit

●

🔗

Figura 2.1.11 - Dictionaries

Exercise

In this exercise, print an alphabetically sorted list of all the functions in the `re` module containing the word `find`.

script.py

```
1 import re
2
3 # Your code goes here
4 find_members = []
5 for x in dir(re):
6     if 'find' in x:
7         find_members.append(x)
8 print(sorted(find_members))
```

IPython Shell

<script.py> output:
['findall', 'finditer']

In [1]: |

Great work!

Solution

Submit

●

🔗

Figura 2.1.12 - Modules and Packages

Exercise

Complete the code below so that it reads three integers from a single line, converts each to an `int`, and prints their sum.

script.py

```
1 # write your code here
2 numbers = input().split()
3 sum = 0
4 for x in numbers:
5     sum = sum + int(x)
6 print("sum is %d" %sum)
```

IPython Shell

DataCamp encountered the following error:
EOF when reading a line

In [1]: |

Solution

Submit

Restart Session

Figura 2.1.13.1 - Input and Output (eroare din cauza ca site-ul nu are stdin)

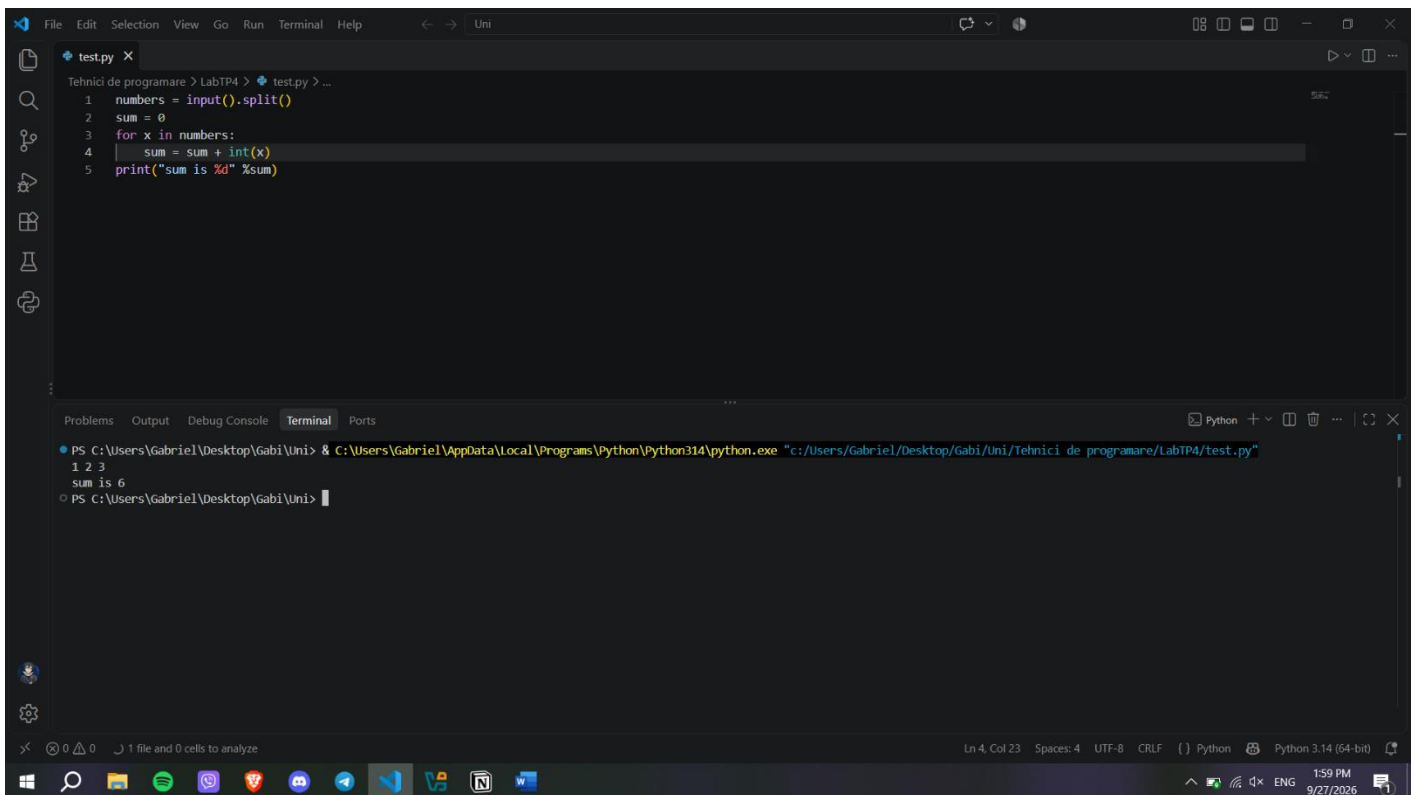


Figura 2.1.13.2 - Input and Output (în VSCode pentru a testa)

2.2. Advanced Tutorials

Write a generator function which returns the Fibonacci series. They are calculated using the following formula: The first two numbers of the series is always equal to 1, and each consecutive number returned is the sum of the last two numbers. Hint: Can you use only two variables in the generator function? Remember that assignments can be done simultaneously. The code will simultaneously switch the values of a and b.

script.py

```
1 # fill in this function
2 def fib():
3     a = 1
4     b = 1
5     for x in range(10):
6         yield a
7         a, b = b, b + a
8     pass #this is a null statement which does
        nothing when executed, useful as a placeholder.
9
10 # testing code
11 import types
12 if type(fib()) == types.GeneratorType:
13     print("Good, The fib function is a generator
        .")
14
15
```

IPython Shell

```
<script.py> output:
Good, The fib function is a generator.
1
1
2
3
5
8
13
21
34
55
```

Good work!

Solution

Submit

Figura 2.2.1 - Generators

Exercise

Using a list comprehension, create a new list called "newlist" out of the list "numbers", which contains only the positive numbers from the list, as integers.

script.py

```
1 numbers = [34.6, -203.4, 44.9, 68.3, -12.2, 44.6,
    12.7]
2 newlist = [x for x in numbers if x > 0]
3 print(newlist)
```

IPython Shell

```
<script.py> output:
[34.6, 44.9, 68.3, 44.6, 12.7]

In [1]:
```

Very nice!

Solution

Submit

Figura 2.2.2 - List Comprehensions

Exercise

Write a program using lambda functions to check if a number in the given list is odd. Print "True" if the number is odd or "False" if not for each element.

script.py

```
1 l = [2,4,7,3,14,19]
2 odd = lambda x : x % 2
3 for i in l:
4     if odd(i) != 0:
5         print("True")
6     else:
7         print("False")
8     # your code here
```

IPython Shell

<script.py> output:
False
False
True
True
False
True

In [1]: |

Nice work!

Solution

Submit

Figura 2.2.3 - Lambda functions

Exercise

Fill in the `foo` and `bar` functions so they can receive a variable amount of arguments (3 or more) The `foo` function must return the amount of extra arguments received. The `bar` must return `True` if the argument with the keyword `magicnumber` is worth 7, and `False` otherwise.

script.py

```
1 # edit the functions prototype and implementation
2 def foo(a, b, c, *rest):
3     return len(rest)
4
5 def bar(a, b, c, **options):
6     return options.get("magicnumber") == 7
7
8
9 # test code
10 if foo(1, 2, 3, 4) == 1:
11     print("Good.")
12 if foo(1, 2, 3, 4, 5) == 2:
13     print("Better.")
14 if bar(1, 2, 3, magicnumber=6) == False:
15     print("Great.")
16 if bar(1, 2, 3, magicnumber=7) == True:
17     print("Awesome!")
```

IPython Shell

<script.py> output:
Good.
Better.
Great.
Awesome!

In [1]: |

Great work!

Solution

Submit

Figura 2.2.4 - Multiple Functions Arguments

script.py

```
8 # End Example
9
10 # Exercise: make a regular expression that will
    match an email
11 def test_email(your_pattern):
12     pattern = re.compile(your_pattern)
13     emails = ["john@example.com", "python
    -list@python.org", "wha.t.`1an?ug{}ly@email.com"]
14     for email in emails:
15         if not re.match(pattern, email):
16             print("You failed to match %s" %
    (email))
17     elif not your_pattern:
18         print("Forgot to enter a pattern!")
19     else:
20         print("Pass")
```

IPython Shell

```
<script.py> output:
    Pass
    Pass
    Pass
In [1]: |
```

Great work!

Solution

Submit

Figura 2.2.5 - Regular Expressions

Exercise

Handle all the exception! Think back to the previous lessons to return the last name of the actor.

script.py

```
1 # Setup
2 actor = {"name": "John Cleese", "rank": "awesome"}
3
4 # Function to modify!!!
5 def get_last_name():
6     try:
7         return actor["last_name"]
8     except:
9         return actor["name"].split()[-1]
10
11 # Test code
12 get_last_name()
13 print("All exceptions caught! Good job!")
14 print("The actor's last name is %s" % get_last_name
    ())
```

IPython Shell

```
<script.py> output:
    All exceptions caught! Good job!
    The actor's last name is Cleese
In [1]: |
```

Great work!

Solution

Submit

Figura 2.2.6 - Exception Handling

In the exercise below, use the given lists to print out a set containing all the participants from event A which did not attend event B.

script.py

```
1 a = set(["Jake", "John", "Eric"])
2 b = set(["John", "Jill"])
3
4 print(a.difference(b))
```

IPython Shell

<script.py> output:
{'Jake', 'Eric'}

In [1]: |

Nice work!

Solution **Submit**

Figura 2.2.7 - Sets

The aim of this exercise is to print out the JSON string with key-value pair "Me" : 800 added to it.

script.py

```
1 import json
2
3 # fix this function, so it adds the given name
4 # and salary pair to salaries_json, and return it
5 def add_employee(salaries_json, name, salary):
6     # Add your code here
7     var = json.loads(salaries_json)
8     var[name] = salary
9     salaries_json = json.dumps(var)
10    return salaries_json
11    return salaries_json
12
13 # test code
14 salaries = '{"Alfred" : 300, "Jane" : 400 }'
15 new_salaries = add_employee(salaries, "Me", 800)
16 decoded_salaries = json.loads(new_salaries)
```

IPython Shell

<script.py> output:
300
400
800

In [1]: |

Great work!

Solution **Submit**

Figura 2.2.8 - Serialization

Exercise

Edit the function provided by calling `partial()` and replacing the first three variables in `func()`. Then print with the new partial function using only one input variable so that the output equals 60.

script.py

```
1 #Following is the exercise, function provided:
2 from functools import partial
3 def func(u, v, w, x):
4     return u*4 + v*3 + w*2 + x
5 #Enter your code here to create and print with your
  partial function
6 func2 = partial(func, 10, 2, 2)
7 print(func2(10))
```

IPython Shell

<script.py> output:
60

In [1]:

Good job!

Solution

Submit

Figura 2.2.9 - Partial Functions

Exercise

Print a list of all attributes of the given Vehicle object.

script.py

```
4 help(hasattr)
5 help(id)
6
7 # Define the Vehicle class.
8 class Vehicle:
9     name = ""
10    kind = "car"
11    color = ""
12    value = 100.00
13    def description(self):
14        desc_str = "%s is a %s %s worth $%.2f." %
15        (self.name, self.color, self.kind, self.value)
16        return desc_str
17 # Print a list of all attributes of the Vehicle
  class.
```

IPython Shell

Return the identity of an object.

This is guaranteed to be unique among
simultaneously existing objects.
(CPython uses the object's memory address.)

```
['__class__', '__delattr__', '__dict__', '__dir__',
'__doc__', '__eq__', '__format__', '__ge__',
'__getattr__', '__gt__', '__hash__',
'__init__', '__le__', '__lt__', '__module__',
'__ne__', '__new__', '__reduce__', '__reduce_ex__',
'__repr__', '__setattr__', '__sizeof__', '__str__',
'__subclasshook__', '__weakref__', 'color',
'description', 'kind', 'name', 'value']
```

Very nice!

Solution

Submit

Figura 2.2.10 - Code Introspection

Exercise

Make a nested loop and a python closure to make functions to get multiple multiplication functions using closures. That is using closures, one could make functions to create `multiply_with_5()` or `multiply_with_4()` functions using closures.

script.py

```
1 # your code goes here
2 def multiplier_of(number):
3     def multiply(factor):
4         return number * factor
5     return multiply
6 multiplywith5 = multiplier_of(5)
7 print(multiplywith5(9))
```

IPython Shell

<script.py> output:
45

In [1]: |

Great work!

Solution

Submit

Figura 2.2.11 - Closures

In this exercise, you'll use each of `map`, `filter`, and `reduce` to fix broken code.

script.py

```
9 my_names = ["olumide", "akinremi", "josiah",
10             "temidayo", "omoseun"]
11 # Use reduce to print the product of these
12   numbers
13 my_numbers = [4, 6, 9, 23, 5]
14 # Fix all three respectively.
15 map_result = list(map(lambda x: round(x ** 2, 3),
16                       my_floats))
17 filter_result = list(filter(lambda name: len(name)
18                             <= 7, my_names))
19 reduce_result = reduce(lambda num1, num2: num1 *
20                        num2, my_numbers, 1)
21 print(map_result)
```

IPython Shell

<script.py> output:
[18.922, 37.088, 10.562, 95.453, 4.666, 78.854, 21.068]
['olumide', 'josiah', 'omoseun']
24840

In [1]: |

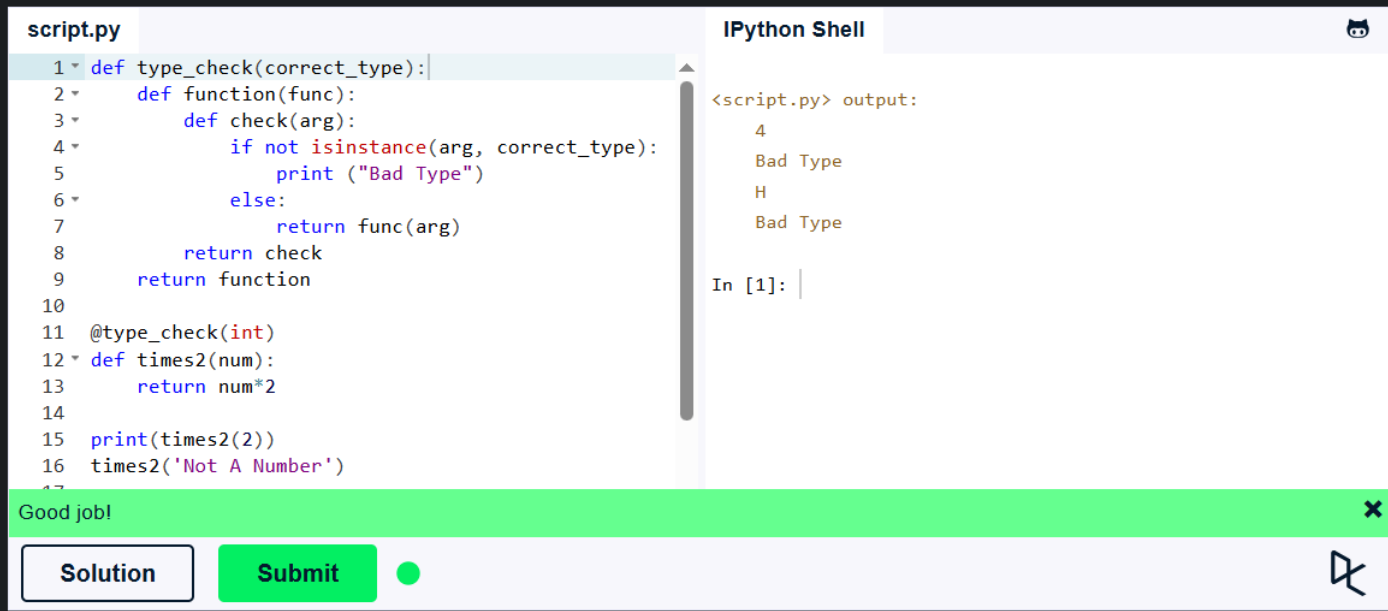
Congrats! Nice work.

Solution

Submit

Figura 2.2.12 - Map, Filter, Reduce

Make a decorator factory which returns a decorator that decorates functions with one argument. The factory should take one argument, a type, and then returns a decorator that makes function should check if the input is the correct type. If it is wrong, it should print("Bad Type") (In reality, it should raise an error, but error raising isn't in this tutorial). Look at the tutorial code and expected output to see what it is if you are confused (I know I would be.) Using `isinstance(object, type_of_object)` or `type(object)` might help.



The screenshot shows a code editor with a file named `script.py` and an adjacent IPython Shell. The script defines a decorator factory `type_check` that takes a type and returns a decorator `check`. The `check` decorator checks if an argument is of the specified type. If not, it prints "Bad Type". The script then uses `@type_check(int)` to decorate a `times2` function and prints the result of `times2(2)` and `times2('Not A Number')`.

```
1 def type_check(correct_type):
2     def function(func):
3         def check(arg):
4             if not isinstance(arg, correct_type):
5                 print("Bad Type")
6             else:
7                 return func(arg)
8         return check
9     return function
10
11 @type_check(int)
12 def times2(num):
13     return num*2
14
15 print(times2(2))
16 times2('Not A Number')
```

The IPython Shell shows the output of the script: `4`, `Bad Type`, `H`, and `Bad Type`. Below the output, it shows the prompt `In [1]:`.

A green notification bar at the bottom says "Good job!". At the bottom of the editor, there are buttons for "Solution" and "Submit", and a small icon on the right.

Figura 2.2.13 - Decorators

CONCLUZII

În concluzie, rezolvarea exercițiilor de pe `codedex.io` și `learnpython.org` a reprezentat o etapă excelentă de pregătire. Am învățat elementele fundamentale ale limbajului python, necesare pentru a continua studiile in domeniul securității informaționale.

BIBLIOGRAFIE

1. Python Tutorial – W3Schools (<https://www.w3schools.com/python/>)
2. Python Tutorial – GeeksforGeeks (<https://www.geeksforgeeks.org/python/python-programming-language-tutorial/>)